

Mini Automated Cyber Security Operations Center (CSOC)

Table of Contents

Contents	Page
Introduction.....	1
System Environment.....	2–5
Wazuh Server Deployment.....	6–7
Agent Deployment.....	8–9
Attack Detection and Testing Using Custom Rules.....	10–29
1.SSH Authentication Failure Detection.....	10–12
2.Ubuntu SSH Brute-Force Detection.....	12–14
3.Windows Brute-Force Detection.....	15–18
4. Windows Firewall Detection.....	19–21
5. PowerShell Monitoring & Detection.....	22–24
6. File Integrity Monitoring (FIM).....	24–27
7. Detection Rule Mapping.....	28
Automatic Incident Response.....	29–34
n8n Automation and Telegram Notification.....	35–43
Security Dashboards.....	44–46
Conclusion.....	47

Mini Automated Cyber Security Operations Center (CSOC)

Introduction

Cyber security is important for protecting computer systems, networks, and data from different types of threats. Security events such as failed login attempts, brute-force attacks, suspicious PowerShell activity, blocked network connections, and file changes need to be monitored and detected quickly.

This project presents a **Mini Automated Cyber Security Operations Center (CSOC)** built in a controlled laboratory environment. **Wazuh** was used as the main security monitoring and detection platform. The project used an Ubuntu Server as the Wazuh Server, a Windows system as the monitored endpoint, and Kali Linux for controlled security testing. n8n was used to automate security-alert processing and send important alerts to Telegram.

Several custom detection rules were created and tested for **SSH authentication failures, SSH brute-force attacks, Windows brute-force activity, Windows Firewall events, suspicious PowerShell Base64 decoding, and File Integrity Monitoring (FIM)**. These tests confirmed that Wazuh could detect the selected security activities and generate the expected alerts.

The project also included an automatic response system for Ubuntu SSH brute-force attacks. When five failed attempts were detected from the same IP within 120 seconds, Wazuh automatically blocked the attacker IP for 120 seconds and then removed the block automatically.

Finally, n8n and Telegram were connected with Wazuh to provide automatic notifications for high-severity alerts. Two Wazuh dashboards were also created to provide a clear view of security alerts, detection events, and response activities.

Overall, this project demonstrates the practical implementation of a compact but functional automated CSOC capable of performing the fundamental activities of a security operations environment: **monitoring, detection, analysis, automated response, notification, and visualization.**

System Environment

The Mini Automated Cyber Security Operations Center (CSOC) was developed in a controlled laboratory environment using **Oracle VirtualBox**. The laboratory consisted of separate systems for security monitoring, endpoint monitoring, and controlled security testing. This separation allowed the project to simulate a practical SOC environment while keeping attack-generation and monitoring activities logically separated.

The main laboratory components were:

1. **Ubuntu Server Virtual Machine** — Central Wazuh SIEM and automation server
2. **Kali Linux Virtual Machine** — Controlled attack and security-testing system
3. **Windows Host** — Monitored endpoint with Wazuh Agent

The detailed system configuration is presented below.

Virtualization Platform:

Oracle VirtualBox was used to create and manage the virtual machines required for the CSOC laboratory.

Configuration	Details
Virtualization Platform	Oracle VirtualBox
Ubuntu VM	Wazuh Server
Kali VM	Security Testing / Attack Simulation

The virtualized environment allowed the Wazuh server and Kali Linux testing system to operate independently while communicating with the Windows endpoint over the laboratory network.

Ubuntu Wazuh Server Virtual Machine:

The primary security-monitoring system was deployed as an Ubuntu Server virtual machine inside VirtualBox.

Parameter	Configuration
Virtual Machine	Ubuntu Wazuh Server
Operating System	Ubuntu Server 24.04.4 LTS
Hostname	wazuh-server
IP Address	192.168.0.243
Architecture	x86-64
Virtualization	Oracle VirtualBox
CPU	4 vCPU
RAM	5 GB
SSH	OpenSSH enabled
Wazuh Version	4.14.6

The Ubuntu VM served as the central security infrastructure of the project. The following components were deployed on this system:

- Wazuh Manager
- Wazuh Indexer
- Wazuh Dashboard
- n8n Automation Platform

- Docker
- Docker Compose

The Wazuh Manager was responsible for receiving and analyzing security events, while the Wazuh Indexer stored the generated alerts and the Wazuh Dashboard provided the interface for monitoring, investigation, and visualization.

The Ubuntu server also hosted the n8n automation platform, which was used to process high-severity Wazuh alerts and forward them to Telegram. The n8n service was exposed through TCP port 5678.

Kali Linux Virtual Machine:

A separate Kali Linux virtual machine was used as the controlled security-testing environment.

Parameter	Configuration
Virtual Machine	Kali Linux
Operating System	Kali Linux
IP Address	192.168.0.191
Virtualization	Oracle VirtualBox
Role	Attack Simulation / Security Testing

Kali Linux was intentionally kept separate from the Wazuh monitoring infrastructure. It was not registered as a Wazuh Agent because its purpose was to generate controlled security-testing activity against the monitored systems.

The Kali system was used during controlled testing of:

- SSH authentication failures
- SSH brute-force activity
- Windows brute-force activity
- TCP SYN scanning against the Windows host

Windows Monitored Endpoint:

The Windows system was used as the primary monitored endpoint of the CSOC environment.

Parameter	Configuration
System	Windows Host
Hostname	biplob
IP Address	192.168.0.155
Wazuh Agent Name	Windows-Host
Wazuh Agent ID	001
Role	Monitored Endpoint
Wazuh Agent	Installed

Unlike Ubuntu Server and Kali Linux, the Windows system was used as the endpoint from which security telemetry was collected.

The Wazuh Agent on Windows was configured to collect several types of security information, including:

- Windows Security Events
- Windows Firewall Events
- PowerShell Operational Events
- File Integrity Monitoring Events

These events were forwarded to the central Wazuh Manager at 192.168.0.243 for analysis and detection. Windows authentication failures.

Network Configuration:

All major systems were connected through the laboratory network using the 192.168.0.0/24 network.

System	IP Address	Role
Ubuntu Wazuh Server	192.168.0.243	Central SIEM / Wazuh Server
Windows Host	192.168.0.155	Monitored Endpoint
Kali Linux	192.168.0.191	Attack Simulation

The network configuration enabled the Kali testing system to generate controlled traffic and authentication attempts toward the Windows and Ubuntu systems, while the Windows Wazuh Agent forwarded endpoint events to the central Wazuh Manager.

Overall CSOC Architecture:

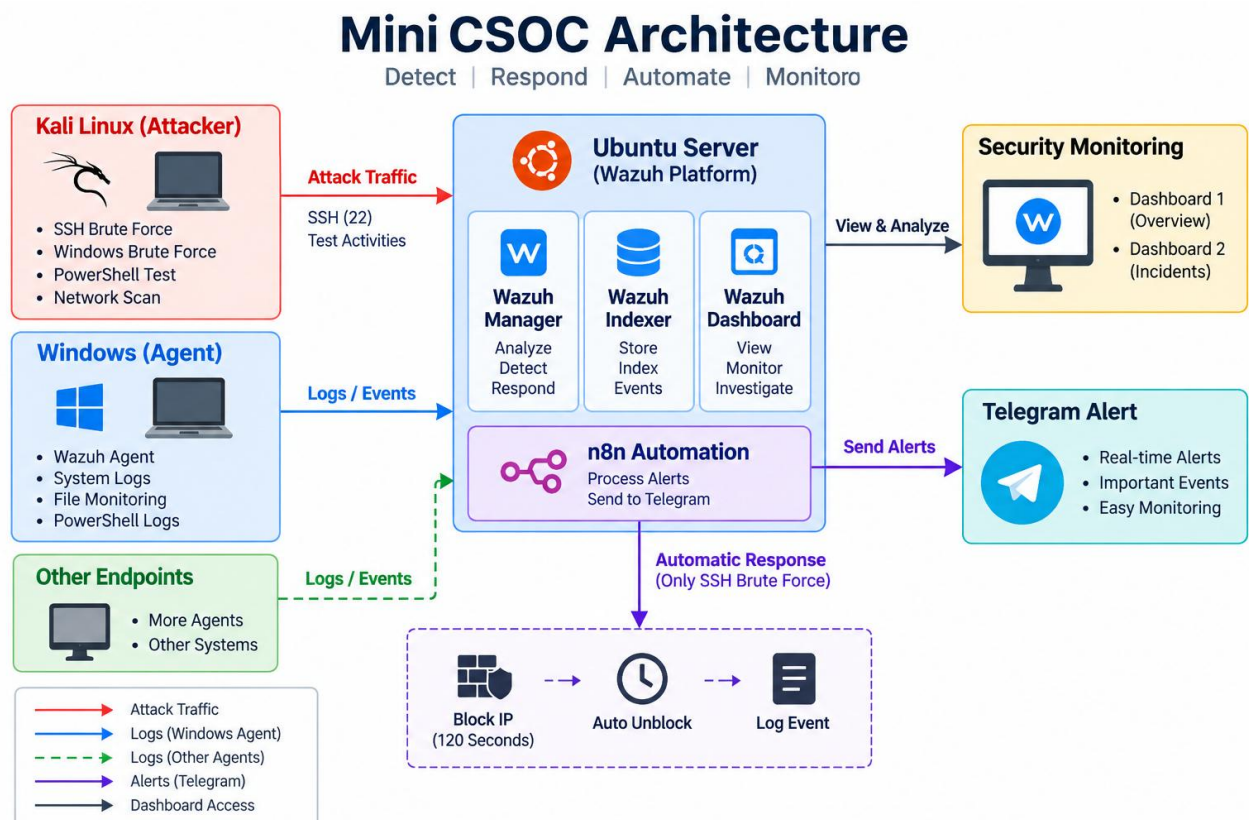


Fig: Overall mini CSOC Architecture

The diagram shows the main workflow of the Mini Automated CSOC. Kali Linux is used for controlled security testing, while the Windows Host sends security logs to the Wazuh Server through the Wazuh

Agent. The Wazuh Manager analyzes events and applies detection rules, and the Wazuh Indexer stores the security data. The Wazuh Dashboard is used to view and investigate alerts. High-severity alerts are sent to n8n, which processes the alerts and sends notifications to Telegram. For Ubuntu SSH brute-force attacks, Wazuh Active Response automatically blocks the attacker IP for 120 seconds and then unblocks it automatically. This workflow provides **detection, monitoring, automation, notification, and response** in one CSOC environment.

Software and Service Distribution:

The final software distribution of the laboratory was:

System	Main Software / Service	Purpose
Ubuntu VM	Ubuntu Server 24.04.4 LTS	Server OS
Ubuntu VM	Wazuh Manager 4.14.6	Event analysis & detection
Ubuntu VM	Wazuh Indexer	Alert storage & indexing
Ubuntu VM	Wazuh Dashboard	Security monitoring
Ubuntu VM	n8n	Alert automation
Ubuntu VM	Docker / Docker Compose	n8n deployment
Kali VM	Kali Linux	Controlled security testing
Windows Host	Wazuh Agent	Endpoint monitoring
Windows Host	Windows Security	Authentication monitoring
Windows Host	Windows Firewall	Network security monitoring
Windows Host	PowerShell	PowerShell activity monitoring
Windows Host	Wazuh FIM	File integrity monitoring
Telegram	MiniCSOC_AlertBot	Security notifications

Wazuh Server Deployment

The Wazuh Server was deployed on the Ubuntu Server virtual machine. The installation was performed using the Wazuh installation script. An All-in-One deployment was used so that the Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard could operate on the same server.

Download and run the Wazuh installation assistant...

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
chmod +x wazuh-install.sh
sudo ./wazuh-install.sh -a
```

The All-in-One deployment installed the main Wazuh server components on the Ubuntu virtual machine:

- **Wazuh Manager**
- **Wazuh Indexer**
- **Wazuh Dashboard**

The Wazuh server was configured on the Ubuntu VM with the following environment:

Parameter	Configuration
Operating System	Ubuntu Server 24.04.4 LTS
Hostname	wazuh-server
IP Address	192.168.0.243
Wazuh Version	4.14.6
Deployment Type	All-in-One

The Wazuh Indexer was subsequently verified to be running successfully on the server.

Wazuh Service Verification:

After completing the installation, the Wazuh services were checked before accessing the Dashboard through a web browser.

```
sudo systemctl status wazuh-manager
sudo systemctl status wazuh-indexer
sudo systemctl status wazuh-dashboard
```

Output is

```
Active
Active
active
```

Wazuh Dashboard Access:

After verifying the Wazuh services, the Dashboard was accessed through a web browser using the Wazuh Server IP address: <https://192.168.0.243> then the wazuh login page open and input my username(admin) and password.

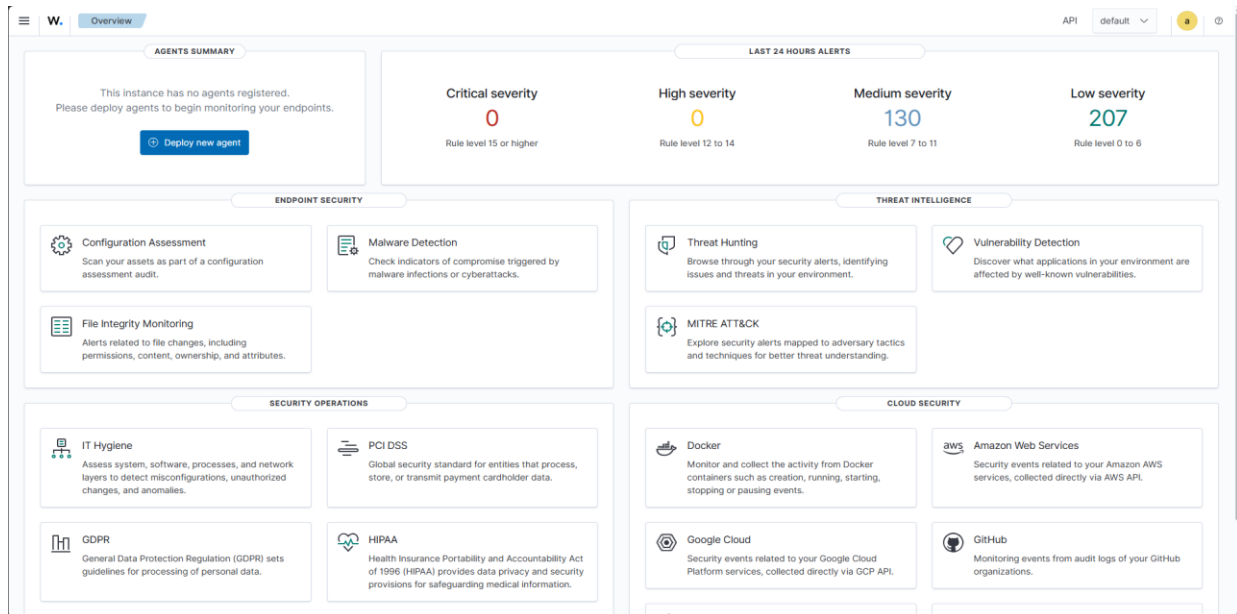


Figure : Main Wazuh Dashboard view

Agent Deployment

Windows Wazuh Agent Deployment:

The Windows endpoint was integrated with the Wazuh Server by deploying the Wazuh Agent through the Wazuh Dashboard web interface. The agent was configured to communicate with the Wazuh Manager at 192.168.0.243.

Open Agent Deployment:

From the Wazuh Dashboard, the Endpoints → Deploy new agent option was selected.

Agent configuration:

Parameter	Configuration
Operating System	Windows
Package	MSI 32/64 bits
Wazuh Server Address	192.168.0.243
Agent Name	Windows-Host

The screenshot shows the 'Deploy new agent' configuration page in the Wazuh Dashboard. The 'Operating System' is set to 'WINDOWS'. Under 'Package', 'MSI 32/64 bits' is selected. The 'Server address' is '192.168.0.243' and the 'Remember server address' checkbox is checked. Under 'Optional settings', the 'Assign an agent name' field is 'Windows-Host'. A warning message at the bottom states: 'The agent name must be unique. It can't be changed once the agent has been enrolled.'

Fig: Windows Agent Deployment Configuration

Download and Install the Agent:

The Wazuh Dashboard generated the Windows agent installation command with the configured server address and agent name.

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi  
-OutFile $env:tmp\wazuh-agent; msixec.exe /i $env:tmp\wazuh-agent /q  
WAZUH_MANAGER='192.168.0.243' WAZUH_AGENT_NAME='Windows-Host'
```

The command was executed from an Administrator PowerShell terminal on the Windows host.

Start the Wazuh Agent:

After installation, the Wazuh Agent service was started using:

```
NET START Wazuh
```

4 Run the following commands to download and install the agent:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi -OutFile $env:temp\wazuh-agent; msexec.exe /s $env:temp\wazuh-agent /q WAZUH_MANAGER="192.168.0.243" WAZUH_AGENT_NAME="Windows-Host"
```

5 Start the agent:

```
NET START Wazuh
```

6 Go to endpoints to verify the agent connection:

[Back to agent list](#)

Fig: Windows Agent Connection Verification

Verify Agent Connection

The agent service was verified from Windows PowerShell:

```
Get-Service WazuhSvc
Status Name DisplayName
-----
Running WazuhSvc Wazuh
```

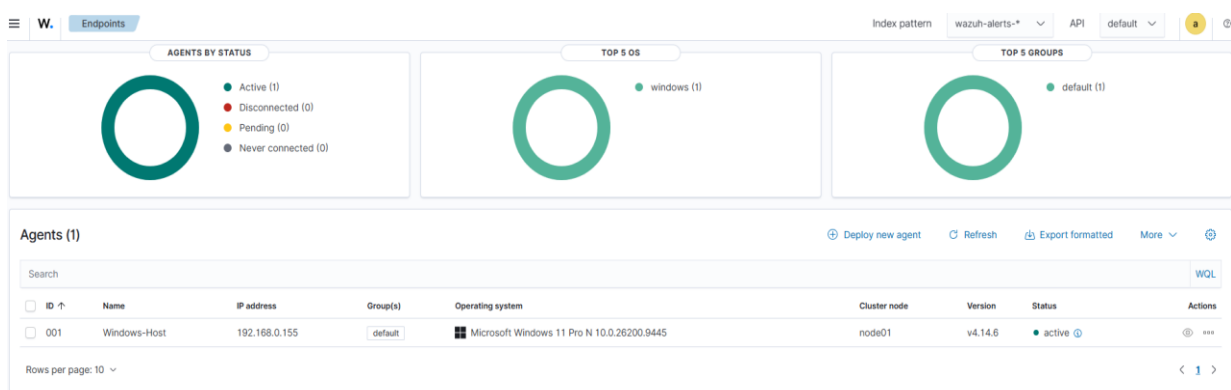


Fig: Agent Status in Wazuh Dashboard

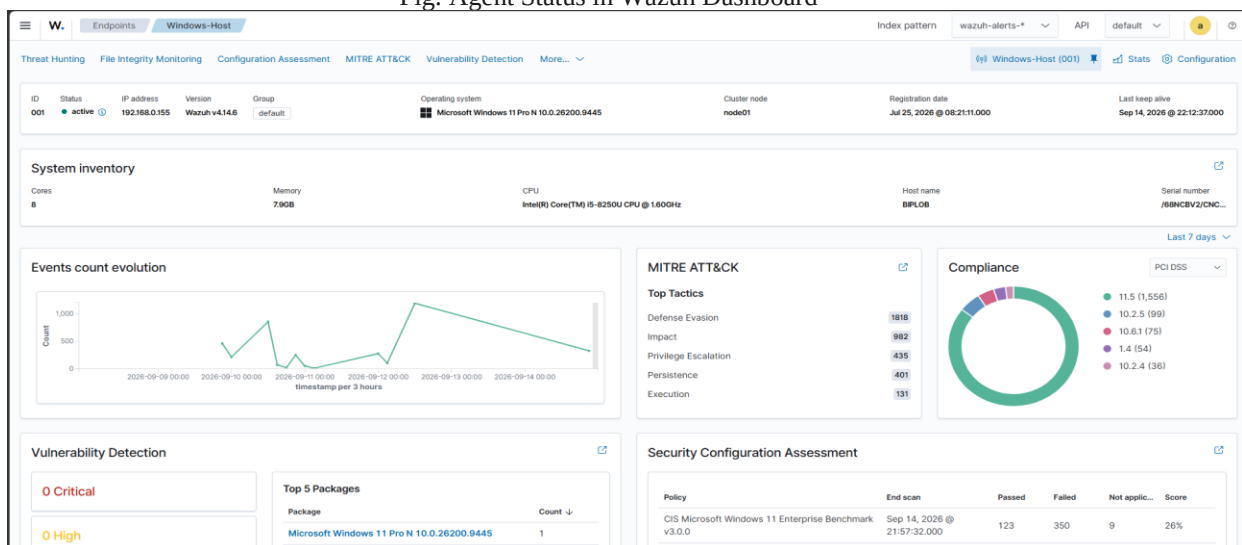


Fig: Windows-Host Endpoint Overview

Attack Detection and Testing Using Custom Rules

The objective of this module was to test and validate the custom Wazuh detection rules by simulating controlled security events, including authentication failures, brute-force attacks, blocked network connections, suspicious PowerShell activity, and file changes. The major detection and testing activities are briefly presented below.

1. SSH Authentication Failure Detection

The objective of this module was to establish baseline visibility for SSH authentication failures on the Ubuntu Wazuh Server. The module was designed to collect SSH authentication logs, process them through Wazuh, and generate a project-specific security alert whenever an unsuccessful SSH authentication attempt was detected. This module provides the foundation for the subsequent SSH brute-force detection mechanism by identifying individual failed authentication events before they are correlated into repeated attack activity.

Log Source Configuration:

The Ubuntu Wazuh server monitors its own SSH authentication log, `/var/log/auth.log` was configured as a local log source in `ossec.conf`

open

```
sudo nano /var/ossec/etc/ossec.conf
```

Inside the configuration, add:

```
<localfile>
  <location>/var/log/auth.log</location>
  <log_format>syslog</log_format>
</localfile>
```

After saving the configuration, the Wazuh Manager was restarted. the configuration

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl is-active wazuh-manager
```

Expected:

```
active
```

Custom Rule 100001:

A custom rule was added in (Go to wazuh → server management → Rules → custom rules → `local_rules.xml`) to provide a project-specific SSH authentication-failure alert, the following custom rule was added to:

```

<group name="local,csoc,sshd,ssh,syslog,authentication_failed">
  <rule id="100001" level="5">
    <if_sid>5760</if_sid>
    <description>CSOC: sshd authentication failed from $(srcip)</description>
    <mitre>
      <id>T1110</id>
    </mitre>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,csoc,</group>
  </rule>
</group>

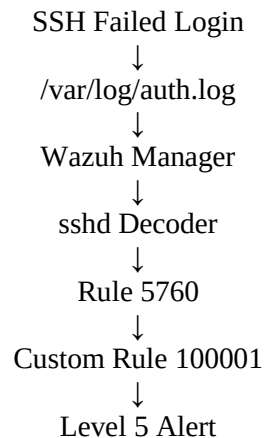
```

Now Save and reload

Rule Explanation:

Parameter	Value
Rule ID	100001
Level	5
Parent Rule	5760
Source Field	srcip
MITRE ATT&CK	T1110
Purpose	SSH authentication failure

Rule Logic:



Test Execution and Results:

To validate the SSH authentication-failure detection mechanism, a controlled failed-login test was performed from the Windows host against the Ubuntu Wazuh Server. The correct username was provided, but an incorrect password was intentionally entered three consecutive times. The SSH client returned authentication failure messages indicating that the login attempts were unsuccessful.

```
C:\Users\biplo>ssh vanced@192.168.0.243
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
vanced@192.168.0.243: Permission denied (publickey,password).
```

Fig: Controlled SSH Authentication-Failure Test

The events were collected by Wazuh from the configured SSH authentication log and processed by the SSH decoder. The built-in Wazuh rule 5760 identified the failed authentication events, after which the custom CSOC Rule 100001 generated a project-specific Level 5 alert.

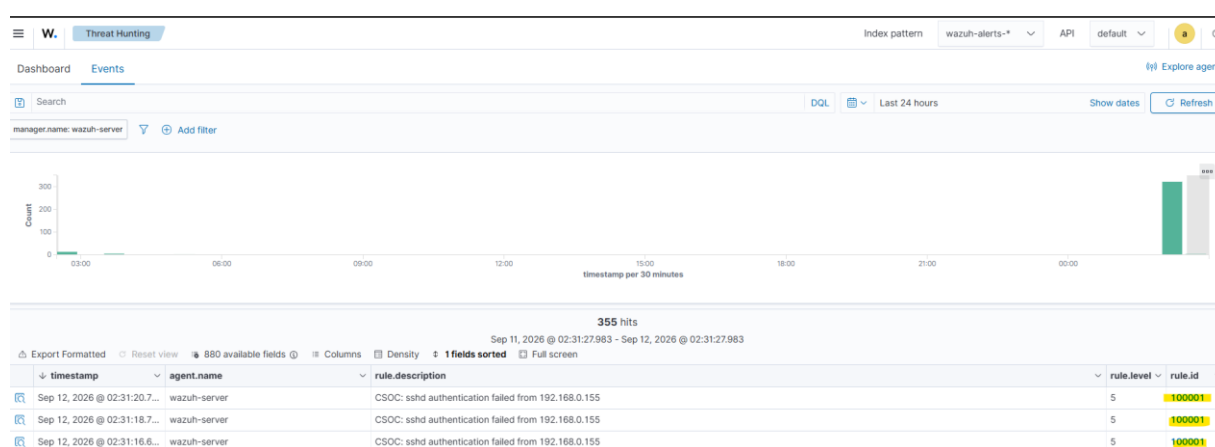


Fig: Wazuh Detection of SSH Authentication Failures

2. Ubuntu SSH Brute-Force Detection

The purpose of this module was to detect repeated SSH authentication failures originating from the same source IP and classify the activity as an Ubuntu SSH brute-force attack. The detection mechanism was implemented using Wazuh custom rules. Individual Ubuntu SSH authentication failures were first identified by the built-in Wazuh rule 5760 and classified by the custom rule 100001. The custom SSH brute-force rule 100101 then correlated repeated authentication-failure events from the same source IP. The final detection logic was configured to identify five or more failed SSH authentication attempts from the same source IP within 120 seconds.

Custom Rule 100101:

A custom rule was added in (Go to wazuh → server management → Rules → custom rules → local_rules.xml) The rule therefore triggers when Wazuh observes five failed Ubuntu SSH authentication events from the same source IP within 120 seconds.

```

<group name="local,csoc,sshd,ssh,syslog,authentication_failed">
<rule id="100101" level="12" frequency="5" timeframe="120">
  <if_matched_sid>100001</if_matched_sid>
  <same_source_ip />
  <description>CSOC ALERT: SSH brute force attack - 5+ failed attempts from
$(srcip) within 120s</description>
  <mitre>
    <id>T1110</id>
  </mitre>
  <group>brute_force,authentication_failures,csoc,</group>
</rule>
</group>

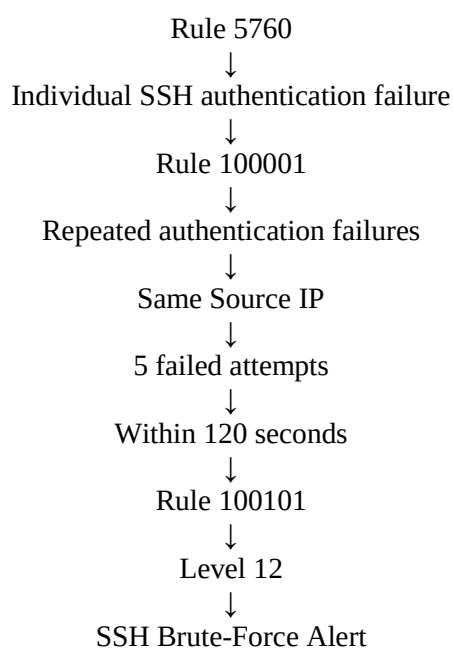
```

Now Save and reload

Rule Parameters:

Parameter	Configuration
Rule ID	100101
Severity Level	12
Parent Event	100001
Base Detection Rule	5760
Frequency	5 events
Timeframe	120 seconds
Source Correlation	Same Source IP
Detection Type	SSH Brute Force
MITRE ATT&CK	T1110 – Brute Force

Rule Logic:



Therefore, Rule 5760 is the original Wazuh detection rule, Rule 100001 identifies each SSH authentication failure as a CSOC event, and Rule 100101 performs the final brute-force correlation.

Test Execution and Results:

Attack source: Kali Linux (192.168.0.191). Target: Ubuntu Wazuh Server (192.168.0.243). Multiple failed SSH authentication attempts were generated from Kali against the Ubuntu host, with events collected from /var/log/auth.log.

The following command was used to generate multiple failed SSH authentication attempts:

```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ ssh vanced@192.168.0.243
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
tPermission denied, please try again.
vanced@192.168.0.243's password:
vanced@192.168.0.243: Permission denied (publickey,password).

(kali@kali)-[~/Desktop]
$ ssh vanced@192.168.0.243
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
```

Fig: Controlled SSH Brute-Force Simulation from Kali Linux

The Wazuh Dashboard successfully detected the repeated SSH authentication failures. Individual failed authentication events were identified by Rule 100001 — Level 5. After the configured threshold of five failed authentication events from the same source IP within 120 seconds was reached, the correlation rule 100101 generated the final brute-force alert.

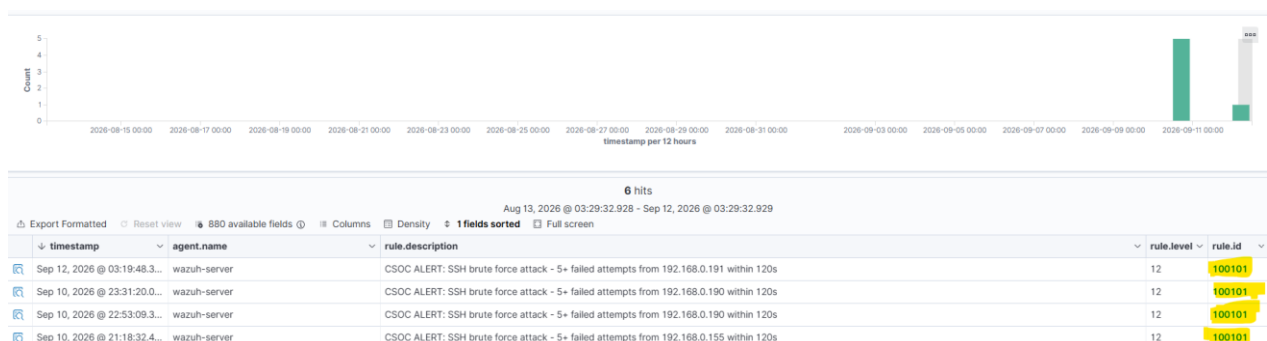


Fig: Wazuh Detection of SSH Brute-Force Activity

3.Windows Brute-Force Detection

The objective of this module was to detect repeated failed login attempts on the Windows host using Wazuh. Failed authentication attempts can be an indication of password guessing or brute-force activity. For the Mini Automated CSOC project, Wazuh was configured to monitor Windows Security events and identify repeated failed authentication attempts. A custom Wazuh rule was then created to detect multiple failed logon events associated with the same target username.

Custom Rule 100100:

A custom rule was added in (Go to wazuh → server management → Rules → custom rules → local_rules.xml) on the Ubuntu Wazuh Server in:

```
<group name="local,csoc,windows,">

  <rule id="100100" level="10" frequency="5" timeframe="120">
    <if_matched_sid>60122</if_matched_sid>
    <same_field>win.eventdata.targetUserName</same_field>
    <description>CSOC ALERT: Windows brute-force attack - multiple failed logons
from $(win.eventdata.targetUserName)</description>
    <mitre>
      <id>T1110</id>
    </mitre>
    <group>authentication_failures,brute_force,csoc,windows,</group>
  </rule>

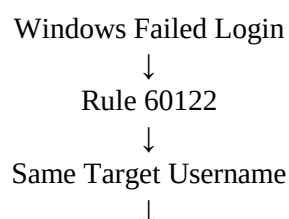
</group>
```

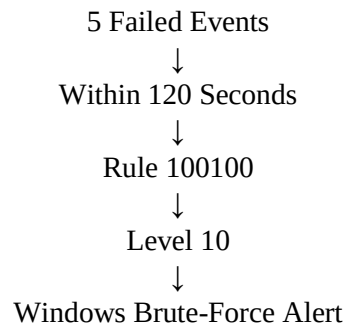
Now Save and reload

Explanation of Rule 100100:

Parameter	Value	Purpose
Rule ID	100100	Custom CSOC detection rule
Level	10	High-severity alert
Frequency	5	Five matching events
Timeframe	120 seconds	Detection window
Parent Rule	60122	Windows failed-login event
Same Field	win.eventdata.targetUserName	Same target username
MITRE	T1110	Brute Force
Group	authentication_failures,brute_force,csoc,windows	Event classification

Rule Logic:





The Windows detection mechanism is based on the built-in Wazuh rule 60122, which identifies Windows logon failures caused by an unknown user or incorrect password. Custom rule 100100 correlates five matching failures within 120 seconds for the same target username.

Test Execution and Results:

The test was executed from an elevated PowerShell session on the Windows host, generating five repeated authentication failures against the local IPC\$ share with an invalid credential:

```
1..5 | ForEach-Object {  
    net use \\127.0.0.1\IPC$ /user:FakeCSOCUser  
    Start-Sleep -Seconds 3  
}
```

Each attempt returned the expected authentication error:

```
PS C:\Windows\system32> 1..5 | ForEach-Object {  
>>     net use \\127.0.0.1\IPC$ /user:FakeCSOCUser  
>>     Start-Sleep -Seconds 3  
>> }  
The password or user name is invalid for \\127.0.0.1\IPC$.  
Enter the password for 'FakeCSOCUser' to connect to '127.0.0.1':  
System error 1326 has occurred.  
  
The user name or password is incorrect.  
  
The password or user name is invalid for \\127.0.0.1\IPC$.  
Enter the password for 'FakeCSOCUser' to connect to '127.0.0.1':  
System error 1326 has occurred.  
  
The user name or password is incorrect.  
  
The password or user name is invalid for \\127.0.0.1\IPC$.  
Enter the password for 'FakeCSOCUser' to connect to '127.0.0.1':  
System error 1326 has occurred.  
  
The user name or password is incorrect.  
  
The password or user name is invalid for \\127.0.0.1\IPC$.  
Enter the password for 'FakeCSOCUser' to connect to '127.0.0.1':  
System error 1326 has occurred.  
  
The user name or password is incorrect.  
  
The password or user name is invalid for \\127.0.0.1\IPC$.  
Enter the password for 'FakeCSOCUser' to connect to '127.0.0.1':
```

Fig: Windows Failed Authentication Attempts

Wazuh's built-in Rule 60122 first identified the individual failed logon events. After five matching events associated with the same target username were observed within the configured 120-second window, the custom Rule 100100 generated the final brute-force alert.

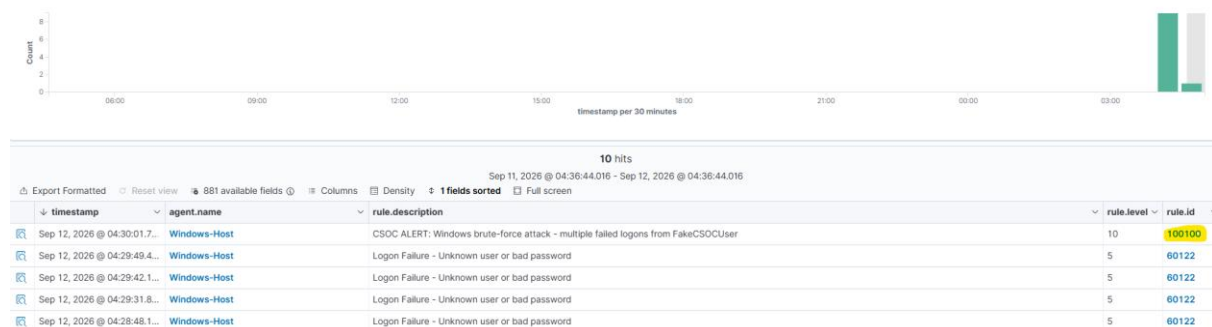


Fig: Wazuh Windows Brute-Force Detection

Windows SSH Failed Login Detection:

The same Windows detection mechanism was observed during failed SSH authentication attempts. The event contained:

```
targetUserName = csocadmin
processName = C:\Windows\System32\OpenSSH\sshd.exe
logonType = 8
```

Wazuh successfully recognized these authentication failures and applied Rule 100100 after the configured threshold was reached.

However, the Windows OpenSSH 4625 event did not provide a usable source IP field in the final alert data. Therefore, the Windows SSH event was used for failed-login/brute-force detection based on the target username, not for automatic source-IP blocking.

Test Execution and Results:

The Windows brute-force detection mechanism was tested by generating repeated failed SSH authentication attempts against the monitored Windows host. The test was performed from the Kali Linux machine using the csocadmin account. Five consecutive SSH connection attempts were made to the Windows host, and an incorrect password was intentionally entered for each attempt. The following command was used:

```

(kali@kali) ~ - [~/Desktop]
$ for i in {1..5}; do
  ssh -o PreferredAuthentications=password \
    -o PubkeyAuthentication=no \
    -o NumberOfPasswordPrompts=1 \
    -o StrictHostKeyChecking=no \
    csocadmin@192.168.0.155
done
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
csocadmin@192.168.0.155's password:
csocadmin@192.168.0.155: Permission denied (publickey,password,keyboard-inter
active).
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
csocadmin@192.168.0.155's password:
csocadmin@192.168.0.155: Permission denied (publickey,password,keyboard-inter
active).
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
csocadmin@192.168.0.155's password:
csocadmin@192.168.0.155: Permission denied (publickey,password,keyboard-inter
active).
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
csocadmin@192.168.0.155's password:
csocadmin@192.168.0.155: Permission denied (publickey,password,keyboard-inter
active).

```

Fig: Kali Linux SSH Brute-Force Test Against Windows

Following the five failed SSH authentication attempts, the generated Windows Security events were collected by the Wazuh Agent and forwarded to the Wazuh Manager for analysis. The built-in Wazuh Rule 60122 first identified the individual failed logon events. After five matching events associated with the same target username were observed within the configured 120-second timeframe, custom Rule 100100 was triggered at Level 10.

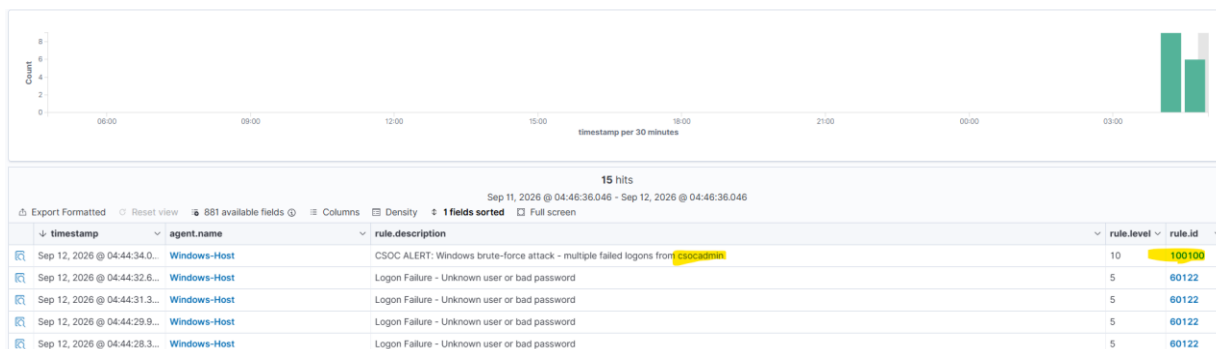


Fig: Wazuh Windows SSH Brute-Force Alert

4. Windows Firewall Detection

The objective of this module was to detect TCP connections blocked by the Windows Firewall and forward the corresponding firewall events to Wazuh for centralized security monitoring and analysis. Windows Firewall logging was enabled on the monitored Windows host, and the generated firewall events were collected by the Wazuh Agent and forwarded to the Wazuh Manager. A custom Wazuh rule, Rule 100200, was then configured to identify blocked TCP connections and generate a CSOC security alert. The detection mechanism was validated using a controlled Nmap TCP SYN scan from the Kali Linux test host against the monitored Windows system.

Firewall Logging Configuration:

This part was done in Administrator PowerShell on Windows.

Run:

```
Set-NetFirewallProfile -Profile Domain,Public,Private -LogBlocked True -  
LogMaxSizeKilobytes 32767
```

Log path: C:\Windows\System32\LogFiles\Firewall\pfirewall.log.

Verify Firewall Log

PowerShell:

```
Get-Content "C:\Windows\System32\LogFiles\Firewall\pfirewall.log" -Tail 20
```

A blocked TCP event appeared similar to:

```
2026-09-07 00:12:51 DROP TCP 192.168.0.191 192.168.0.155 37878 22 60 S ...
```

- Action: DROP
- Protocol: TCP
- Source IP: IP address of the originating host
- Destination IP: 192.168.0.155
- Destination Port: 22

Custom Rule 100200:

A custom rule was added in (Go to wazuh → server management → Rules → custom rules → local_rules.xml) A grouped rule was added on the Ubuntu manager to alert on TCP connections blocked by Windows Firewall, built on top of the built-in decoder rule 4101:

```
<group name="local,csoc,windows,firewall,network_scan,">  
  
  <rule id="100200" level="5">  
    <if_sid>4101</if_sid>
```

```

<protocol>TCP</protocol>
<description>CSOC: TCP connection blocked by Windows Firewall</description>
<group>firewall_block,network_activity,csoc,windows,</group>
</rule>

</group>

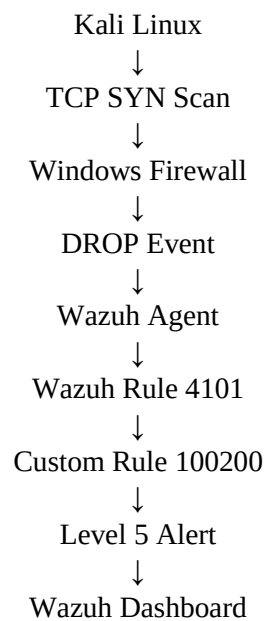
```

Now Save and reload

Rule Parameters:

Parameter	Value
Rule ID	100200
Level	5
Parent Rule	4101
Protocol	TCP
Purpose	Detect blocked TCP connections
Category	Firewall / Network Activity

Rule Logic:



Test Execution and Results:

The Windows Firewall detection mechanism was validated using a controlled TCP SYN scan from the Kali Linux test host against the monitored Windows host. The scan therefore identified 4999 filtered TCP ports, while TCP port 22 was accessible. The filtered TCP connection attempts generated blocked network traffic at the Windows Firewall. These events were recorded in the Windows Firewall log and subsequently collected by the Wazuh Agent.

```
(kali@kali)-[~/Desktop]
$ sudo nmap -sS -p 1-5000 192.168.0.155
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-11 17:52 -0400
Nmap scan report for 192.168.0.155
Host is up (0.0010s latency).
Not shown: 4999 filtered tcp ports (no-response)
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 48:5F:99:82:92:1F (Cloud Network Technology (Samoa) Limited)

Nmap done: 1 IP address (1 host up) scanned in 20.97 seconds
```

Fig: Controlled Nmap TCP SYN Scan Against Windows

The Wazuh Manager successfully decoded the Windows Firewall events using the built-in firewall detection rule 4101. The custom rule 100200 then matched the TCP firewall event and generated a CSOC security alert.

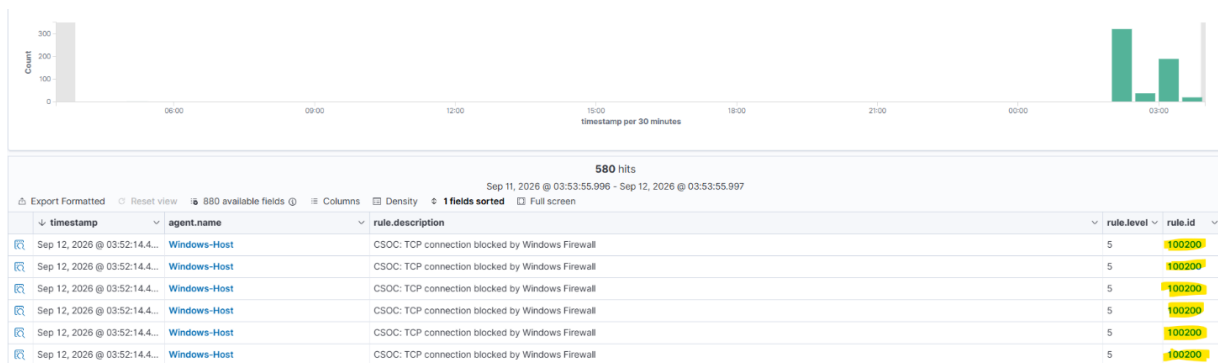


Fig: Wazuh Detection of Blocked TCP Connections

5. PowerShell Monitoring & Detection

Log Source Configuration:

The Windows Wazuh agent (C:\Program Files (x86)\ossec-agent\ossec.conf) was configured to collect the PowerShell Operational event channel:

Open PowerShell as Administrator.

```
notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"
```

Inside the <ossec_config> section, configure:

```
<localfile>
  <location>Microsoft-Windows-PowerShell/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

Save the file in Notepad

Script Block Logging:

PowerShell Script Block Logging was enabled via the registry to capture executed script content (Event ID 4104):

Open Administrator PowerShell.

```
New-Item -Path
"HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Force |
Out-Null
Set-ItemProperty -Path
"HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Name
EnableScriptBlockLogging -Type DWord -Value 1
```

The Windows Wazuh agent service was restarted to apply the configuration change:

```
Restart-Service WazuhSvc
Get-Service WazuhSvc
```

Expected:

Status	Name	DisplayName
Running	WazuhSvc	Wazuh

Custom Rule 100300:

A custom rule was added in (Go to wazuh → server management → Rules → custom rules → local_rules.xml) to flag suspicious Base64 decoding activity in PowerShell, built on top of Wazuh's built-in detection (rule 91809) and mapped to MITRE T1140 (Deobfuscate/Decode Files or Information):

```

<group name="local,csoc,windows,powershell,">

  <rule id="100300" level="12">
    <if_sid>91809</if_sid>
    <description>CSOC ALERT: Suspicious PowerShell Base64 decoding
detected</description>
    <mitre>
      <id>T1140</id>
    </mitre>
    <group>powershell,suspicious_activity,csoc,windows,</group>
  </rule>

</group>

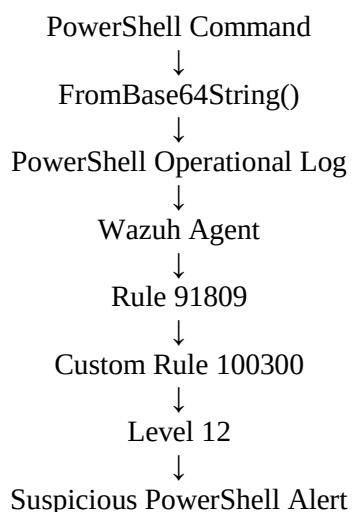
```

Now Save and reload

Rule Parameters:

Parameter	Value
Rule ID	100300
Level	12
Parent Rule	91809
MITRE	T1140 – Deobfuscate/Decode Files or Information
Purpose	Suspicious PowerShell Base64 decoding

Rule Logic:



Test Execution and Results:

The following command was executed to simulate Base64-encoded PowerShell activity:

```
powershell.exe -NoProfile -Command "[Convert]::FromBase64String('SGVsbG8=')
```

```
PS C:\Windows\system32> powershell.exe -NoProfile -Command "[Convert]::FromBase64String('SGVsbG8=')"
```

Fig: PowerShell Base64 Decoding Test

Wazuh's built-in rule 91809 identified the Base64 decoding call, and custom Rule 100300 fired at Level 12.

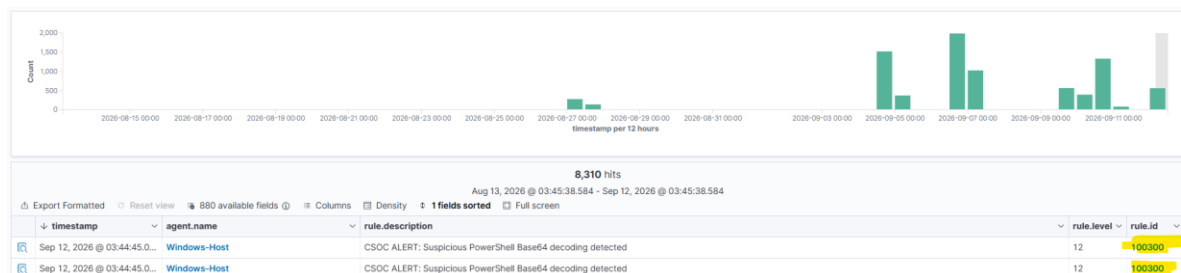


Fig: Wazuh Suspicious PowerShell Alert

6. File Integrity Monitoring (FIM)

The objective of this module was to monitor files on the Windows endpoint and detect file creation, modification, and deletion activities in real time. Wazuh File Integrity Monitoring (FIM) was used to provide visibility into changes made to the monitored files.

Configuration:

The Wazuh Agent already had Syscheck/FIM enabled:

```
<syscheck>
  <disabled>no</disabled>
```

The existing configuration also included real-time monitoring for selected Windows directories.

A dedicated test directory was created on the Windows host:

```
C:\CSOC-FIM-Test
```

The directory was configured for real-time monitoring through the Wazuh Agent:

Run Windows PowerShell Administrator

```
<directories realtime="yes">C:\CSOC-FIM-Test</directories>
```

find the line

```
<directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start  
Menu\Programs\Startup</directories>
```

And add the next line

```
<directories realtime="yes">C:\CSOC-FIM-Test</directories>
```

Like this

```
<directories realtime="yes">%PROGRAMDATA%\Microsoft\Windows\Start  
Menu\Programs\Startup</directories>  
<directories realtime="yes">C:\CSOC-FIM-Test</directories>
```

Save and close notepad

The realtime="yes" option enables Wazuh to detect file changes as they occur.

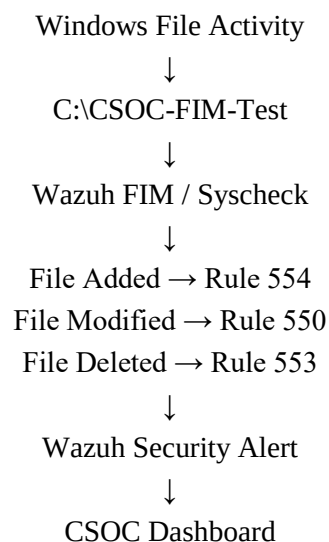
Run Windows PowerShell Administrator

```
Restart-Service WazuhSvc  
Get-Service WazuhSvc
```

Successful output:

Status	Name	DisplayName
-----	----	-----
Running	WazuhSvc	Wazuh

Rule Logic:



Test Execution and Results:

After completing the configuration, file operations were performed inside the monitored directory to verify that Wazuh could detect different types of file changes.

1. File Creation Detection

The monitored directory was created : using PowerShell

```
New-Item -ItemType Directory -Path "C:\CSOC-FIM-Test" -Force
```

A test file was then created:

```
Set-Content -Path "C:\CSOC-FIM-Test\csoc-test.txt" -Value "CSOC FIM Test - Initial"
```

Wazuh successfully detected the new file:

Rule 554 — File added

Result: File creation was successfully detected.

2. File Modification Detection

The contents of the same file were modified: using PowerShell

```
Add-Content -Path "C:\CSOC-FIM-Test\csoc-test.txt" -Value " - Modified by CSOC test"
```

Wazuh detected the change and generated:

Rule 550 — Integrity checksum changed

Result: File modification was successfully detected.

3. File Deletion Detection

Finally, the test file was deleted: using PowerShell

```
Remove-Item "C:\CSOC-FIM-Test\csoc-test.txt"
```

Wazuh successfully detected the deletion:

Rule 553 — File deleted

Result: File deletion was successfully detected.

Operation	Action Simulated	Wazuh Rule	Result
File Creation	New test file added to C:\CSOC-FIM-Test	554 — File added	Detected
File Modification	Content appended to existing test file	550 — Checksum changed	Detected
File Deletion	Test file removed from monitored directory	553 — File deleted	Detected

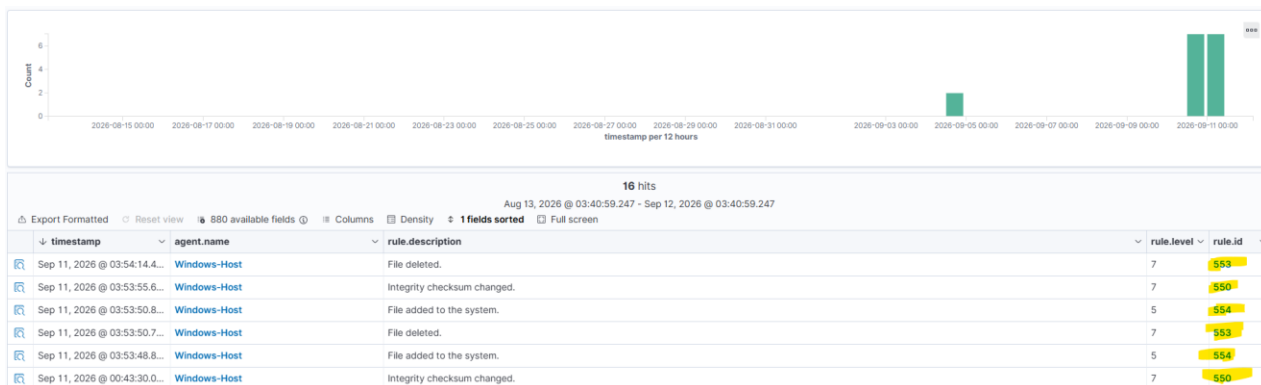


Fig: Wazuh Windows FIM Events Detection

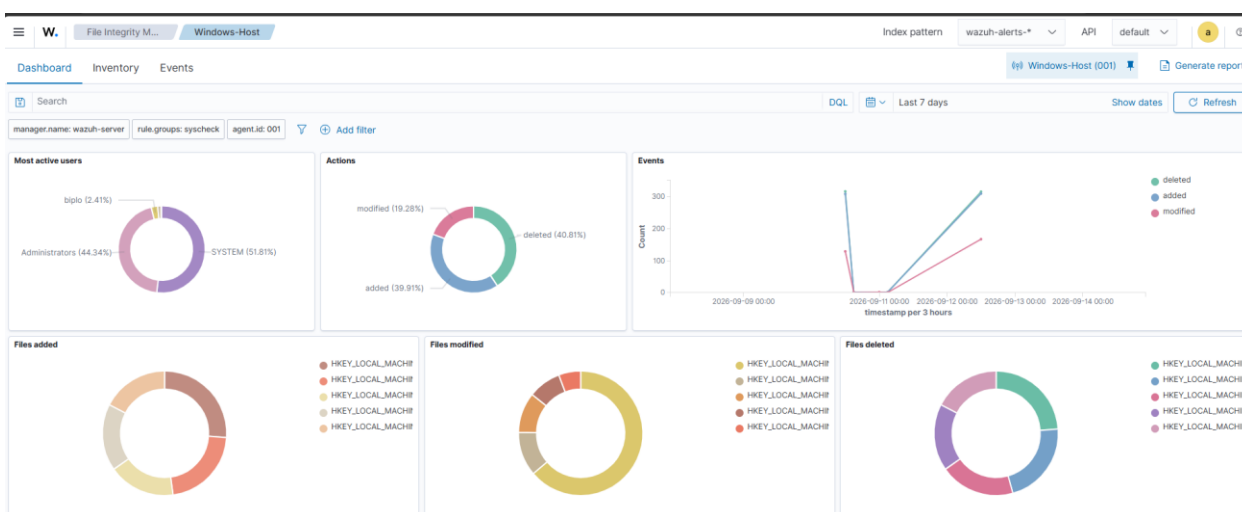


Fig: Windows FIM Events in Wazuh Dashboard

7. Detection Rule Mapping

The table below summarizes every detection module engineered for this project, linking the built-in Wazuh rule used for baseline event parsing to the final custom correlation rule that produces the CSOC alert.

Module	Configuration Location	Built-in Rule	Custom Rule	Level
SSH Authentication Failure	Ubuntu /var/log/auth.log	5760	100001	5
Ubuntu SSH Brute Force	Ubuntu /var/log/auth.log	5760	100101	12
Windows Brute Force	Windows Security Events	60122	100100	10
Windows Firewall	pfirewall.log	4101	100200	5
PowerShell	PowerShell Operational	91809	100300	12
FIM — File Added	Windows Syscheck	554		5
FIM — File Modified	Windows Syscheck	550		7
FIM — File Deleted	Windows Syscheck	553		5

Automatic Incident Response

The Mini Automated CSOC implements an automated incident response mechanism using Wazuh Active Response. The purpose of this component is to automatically detect an SSH brute-force attack, identify the attacker's source IP address, temporarily block the source IP, and automatically remove the block after a predefined period.

In the final implementation, automatic blocking is enabled only for SSH brute-force attacks against the Ubuntu Wazuh Server. Windows security events are monitored and reported by Wazuh, but automatic blocking is intentionally not enabled for Windows.

The final response cycle is:

Detect → Block → Monitor → Automatically Unblock → Recover

SSH Brute-Force Detection Configuration:

The Active Response mechanism depends on the SSH brute-force detection module implemented previously. Individual SSH authentication failures are first identified by the built-in Wazuh Rule 5760. Custom Rule 100001 generates a project-specific Level 5 alert for each failed SSH authentication attempt. Custom Rule 100101 then correlates repeated failed authentication events from the same source IP.

The final Rule 100101 configuration was:

```
<group name="local,csoc,sshd,ssh,syslog,authentication_failed">
<rule id="100101" level="12" frequency="5" timeframe="120">
  <if_matched_sid>100001</if_matched_sid>
  <same_source_ip />
  <description>CSOC ALERT: SSH brute force attack - 5+ failed attempts from
$(srcip) within 120s</description>
  <mitre>
    <id>T1110</id>
  </mitre>
  <group>brute_force,authentication_failures,csoc,</group>
</rule>
</group>
```

Rule Parameters:

Parameter	Configuration
Rule ID	100101
Severity Level	12
Parent Event	100001
Base Detection Rule	5760
Frequency	5 events
Timeframe	120 seconds
Source Correlation	Same Source IP
Detection Type	SSH Brute Force
MITRE ATT&CK	T1110 – Brute Force

Therefore, when five or more failed SSH authentication attempts are detected from the same source IP within 120 seconds, Rule 100101 is triggered.

SSH Authentication Failure Detection:

Previous we configure SSH Authentication Failure Detection using Wazuh configuration. the Ubuntu Wazuh server monitors its own SSH authentication log, /var/log/auth.log was configured as a local log source in ossec.conf. The final Wazuh configuration included:

```
<localfile>
  <location>/var/log/auth.log</location>
  <log_format>syslog</log_format>
</localfile>
```

Wazuh decoded the event as an SSH authentication failure and provided the source IP through: data.srcip. This source IP information is required for the Active Response mechanism because the firewall must know which IP address should be blocked.

Active Response Configuration:

After completing the SSH brute-force detection configuration, Wazuh Active Response was configured on the Ubuntu Wazuh Server.

The Wazuh Manager configuration file was opened using:

```
sudo nano /var/ossec/etc/ossec.conf
```

The following Active Response configuration was added:

```
<active-response>
  <disabled>no</disabled>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100101</rules_id>
  <timeout>120</timeout>
</active-response>
```

Configuration Explanation:

Parameter	Value	Purpose
disabled	no	Enables Active Response
command	firewall-drop	Blocks the source IP
location	local	Executes response on the Wazuh Server
rules_id	100101	Response is triggered by SSH brute-force rule
timeout	120	Block remains for 120 seconds

The important point is that only Rule 100101 is connected to this automatic blocking mechanism. Therefore, the response is not triggered by normal SSH failures, Windows brute-force alerts, Windows Firewall alerts, or PowerShell alerts.

the manager restarted After saving, the configuration

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl is-active wazuh-manager
```

Expected:

```
active
```

Active Response Block Event - Rule 100600:

To make the automatic blocking activity visible in the Wazuh Dashboard, a custom rule (Go to wazuh → server management → Rules → custom rules → local_rules.xml) was configured for the Wazuh Active Response block event.

The final configuration was:

```
<rule id="100600" level="10" overwrite="yes">
  <if_sid>651</if_sid>
  <description>CSOC ALERT: Attacker IP has been BLOCKED by firewall-drop for 120
seconds</description>
  <mitre>
    <id>T1562.004</id>
  </mitre>
  <group>active_response,firewall_drop,csoc,ossec,</group>
</rule>
```

Now Save and reload

Rule Explanation:

Parameter	Value
Rule ID	100600
Level	10
Parent Rule	651
Function	Reports IP blocking
Response Mechanism	firewall-drop
MITRE ATT&CK	T1562.004

Active Response Unblock Event - Rule 100601:

Because the IP block was configured with a timeout, Wazuh automatically removes the temporary block after 120 seconds. A second custom rule (Go to wazuh → server management → Rules → custom rules → local_rules.xml) was configured to make this automatic recovery event visible.

The final configuration was:

```
<rule id="100601" level="10" overwrite="yes">
  <if_sid>652</if_sid>
  <description>CSOC ALERT: Attacker IP has been UNBLOCKED by firewall-drop Active
Response</description>
  <group>active_response,firewall_drop,csoc,ossec,</group>
</rule>
```

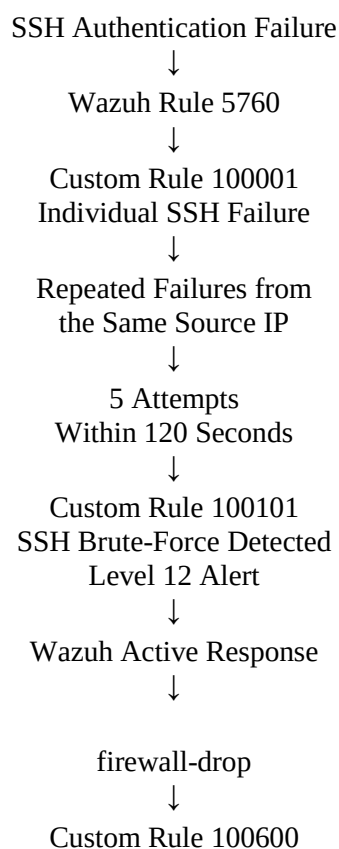
Now Save and reload

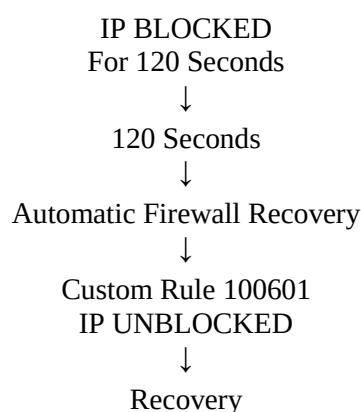
Rule Explanation:

Parameter	Value
Rule ID	100601
Level	10
Parent Rule	652
Function	Reports automatic IP unblocking
Timeout	120 seconds
Response Status	Automatic recovery

Active Response Rules logic:

The complete automated incident response mechanism follows a sequential detection, response, and recovery process. The workflow begins with an individual SSH authentication failure and ends with the automatic recovery of the protected system.





Active Response-related configuration therefore consisted of three important rules

Rule ID	Level	Function
100101	12	Detects SSH brute-force attack
100600	10	Reports attacker IP blocked
100601	10	Reports attacker IP automatically unblocked

Test Execution and Results:

Multiple SSH connection attempts were initiated from Kali Linux against the Ubuntu Wazuh Server. During each connection attempt, an incorrect password was intentionally provided to generate failed SSH authentication events. The incorrect password was entered repeatedly within the configured detection window. These repeated authentication failures generated SSH authentication events on the Ubuntu Wazuh Server.

```

kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ ssh vanced@192.168.0.243
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
tPermission denied, please try again.
vanced@192.168.0.243's password:
vanced@192.168.0.243: Permission denied (publickey,password).

(kali@kali)-[~/Desktop]
$ ssh vanced@192.168.0.243
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
  
```

Fig: Controlled SSH Brute-Force Test for Active Response

The controlled SSH brute-force test successfully validated the complete automated incident response workflow of the Mini Automated CSOC. During the test, multiple failed SSH authentication attempts were generated from the source IP 192.168.0.191 against the Ubuntu Wazuh Server. The individual authentication failures were successfully detected by custom Rule 100001 at Level 5.

After five failed authentication attempts from the same source IP were detected within the configured 120-second timeframe, custom Rule 100101 was triggered at Level 12 with the following alert. Because Rule 100101 was connected to Wazuh Active Response, the firewall-drop command was automatically executed. The blocking activity was reported through custom Rule 100600. After the configured 120-second timeout, the temporary firewall block was automatically removed. The recovery event was successfully detected by custom Rule 100601

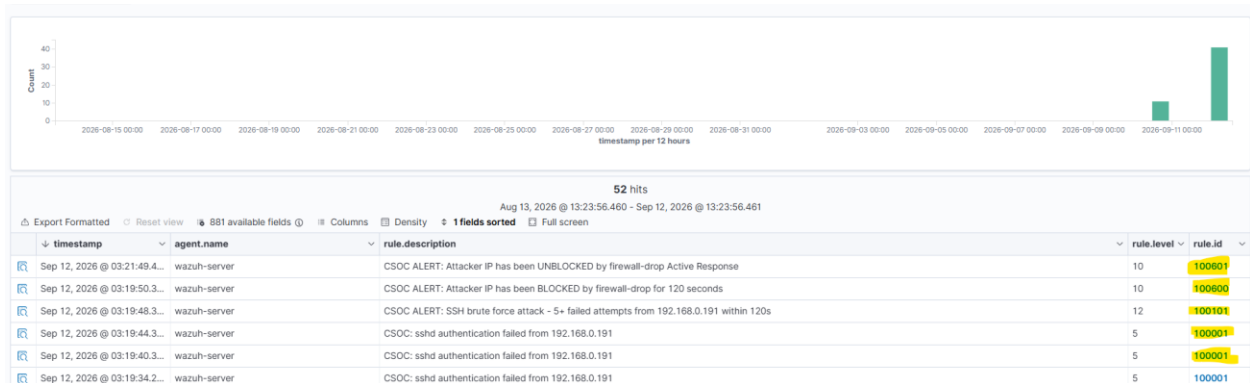


Fig: Complete SSH Brute-Force Detection and Response Cycle

n8n Automation and Telegram Notification

The objective of this component was to automate the security-alert notification process of the Mini Automated CSOC. n8n was deployed as the automation platform and integrated with Wazuh so that high-severity security alerts could be received, filtered, processed, and forwarded automatically to a Telegram bot. The final workflow implements the following sequence:

Wazuh Alert → n8n Webhook → Severity Filter → Alert Data Processing → Telegram Notification.

Docker And n8n Configuration:

n8n was deployed on the Ubuntu Wazuh Server using Docker. The Wazuh server IP address was 192.168.0.243, and n8n was exposed on TCP port 5678. The n8n data directory was created under the Ubuntu user's home directory. Docker was installed on the Ubuntu Wazuh Server using the following commands:

```
sudo apt update && sudo apt install -y docker.io
sudo systemctl enable --now docker
sudo systemctl is-active docker
```

The successful verification output was:

```
active
```

Docker Compose was installed to manage the n8n container configuration:

```
sudo apt install -y docker-compose
docker-compose version
```

The installed version was:

```
docker-compose version 1.29.2, build unknown
```

A dedicated directory for the n8n deployment was created on the Ubuntu Wazuh Server:

```
mkdir -p ~/n8n
cd ~/n8n
```

The deployment directory was:

```
/home/vanced/n8n
```

Before deploying n8n, TCP port 5678 was checked to ensure that it was available:

```
sudo ss -tulpn | grep :5678
```

No process was listening on port 5678, confirming that the port was free for the n8n service.

The n8n container was configured through a docker-compose.yml file.

```
nano ~/n8n/docker-compose.yml
```

The final working configuration was:

```

version: "3.8"

services:
  n8n:
    image: docker.n8n.io/n8nio/n8n:latest
    container_name: n8n
    restart: unless-stopped

    ports:
      - "5678:5678"

    environment:
      - TZ=Asia/Dhaka
      - N8N_SECURE_COOKIE=false
      - N8N_HOST=192.168.0.243
      - N8N_PORT=5678
      - N8N_PROTOCOL=http

    volumes:
      - n8n_data:/home/node/.n8n

volumes:
  n8n_data:

```

The configuration defined the n8n Docker image, container name, restart policy, network port, server address, timezone, and persistent data volume. The n8n_data volume was used to preserve n8n configuration and workflow data.

The n8n container was started from the deployment directory:

```

cd ~/n8n
sudo docker-compose up -d
sudo docker ps

```

The Docker container was successfully started and the port mapping was established as 0.0.0.0:5678->5678/tcp. The n8n web interface was then accessed from a browser using the Wazuh server address: <http://192.168.0.243:5678>. An n8n owner account was created during the initial setup. The Community Edition environment was used, and the optional AI connection was skipped.

n8n Workflow Configuration:

The final n8n workflow was designed as a four-stage automation pipeline consisting of a Webhook, IF, Edit Fields, and Telegram node. The workflow receives Wazuh JSON alerts and forwards only alerts with severity level 10 or higher.

The Webhook node was configured with the following settings:

```

HTTP Method: POST
Authentication: None
Response: Respond Immediately
Path: d988905a-3793-4db8-9d84-f853ba0a6009

```

The production webhook endpoint used by Wazuh was:

Test URL

<http://192.168.0.243:5678/webhook-test/d988905a-3793-4db8-9d84-f853ba0a6009>

Production URL

http://192.168.0.243:5678/webhook/d988905a-3793-4db8-9d84-f853ba0a6009

The webhook was designed to receive Wazuh security alerts in JSON format.

The webhook was tested from the Ubuntu server using a JSON POST request:

```
curl -X POST "http://127.0.0.1:5678/webhook-test/d988905a-3793-4db8-9d84-f853ba0a6009" \  
-H "Content-Type: application/json" \  
-d '{"rule":{"id":"100100","level":10,"description":"CSOC TEST - Possible Windows brute-force attack"},"agent":{"name":"Windows-Host","id":"001"},"data":{"srcip":"192.168.0.191"}}'
```

The webhook successfully received the request and returned one item for processing.

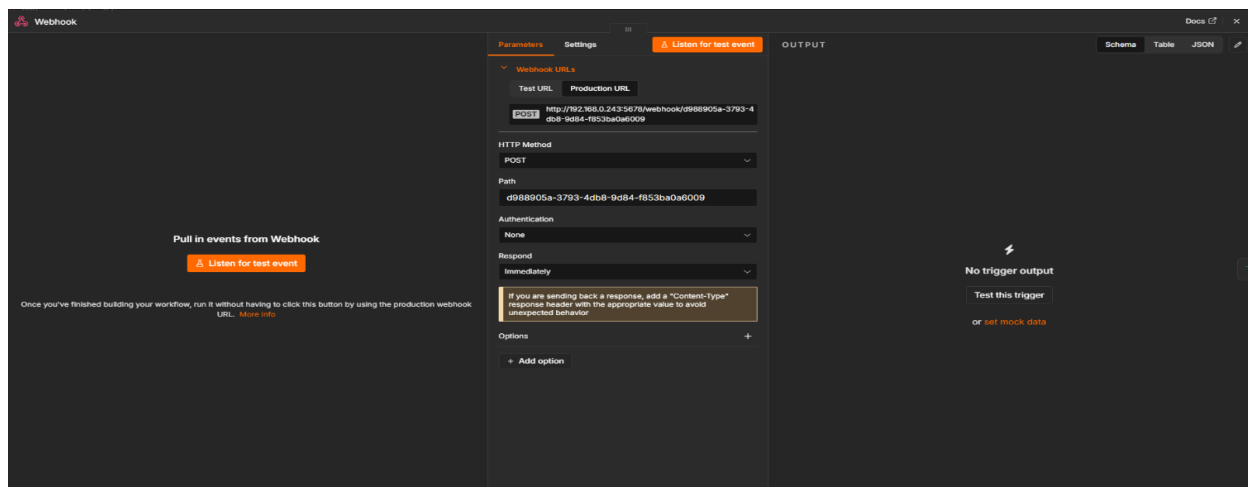


Fig: n8n Webhook Configuration and Test

An IF node was added to ensure that only high-severity Wazuh alerts were forwarded to Telegram. The rule level was evaluated using the following expression:

```
Value 1:  
{{ $json.body.rule.level }}
```

The condition was configured as:

Operator: Greater than or equal to

Value: 10

Alerts satisfying the condition were sent through the True branch, while alerts below severity level 10 were sent through the False branch and were not forwarded to Telegram.

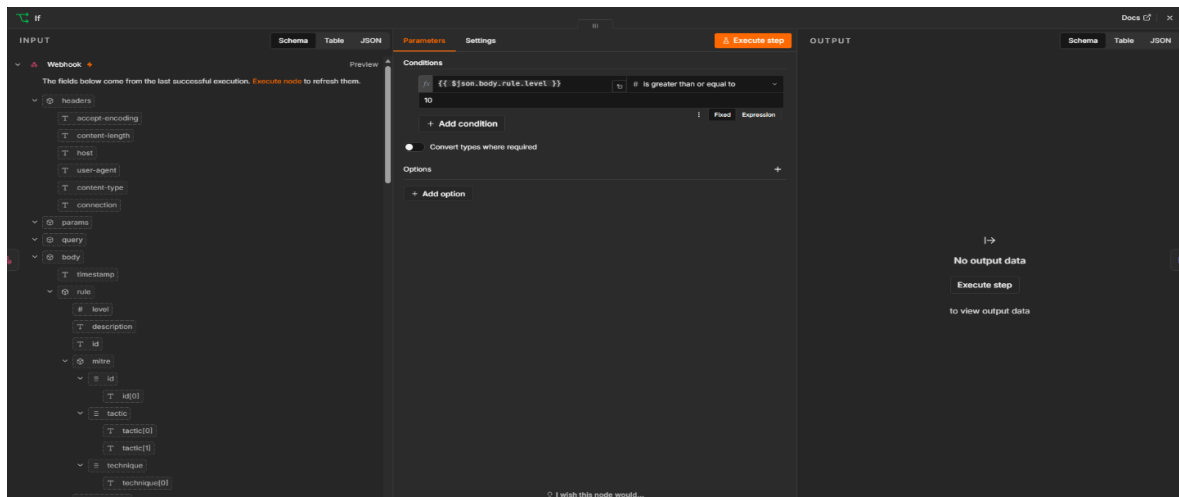


Fig: n8n IF Fields Configuration

The Edit Fields node was configured in Manual Mapping mode to extract the main security-alert fields required for the Telegram message.

```

alert_level
{{ $json.body.rule.level }}
rule_id
{{ $json.body.rule.id }}
agent_name
{{ $json.body.agent.name }}
source_ip
{{ $json.body.data.win?.eventdata?.ipAddress || $json.body.data.srcip || '' }}
description
{{ $json.body.rule.description }}

```

The option to include other input fields was disabled so that only the required fields were passed forward.

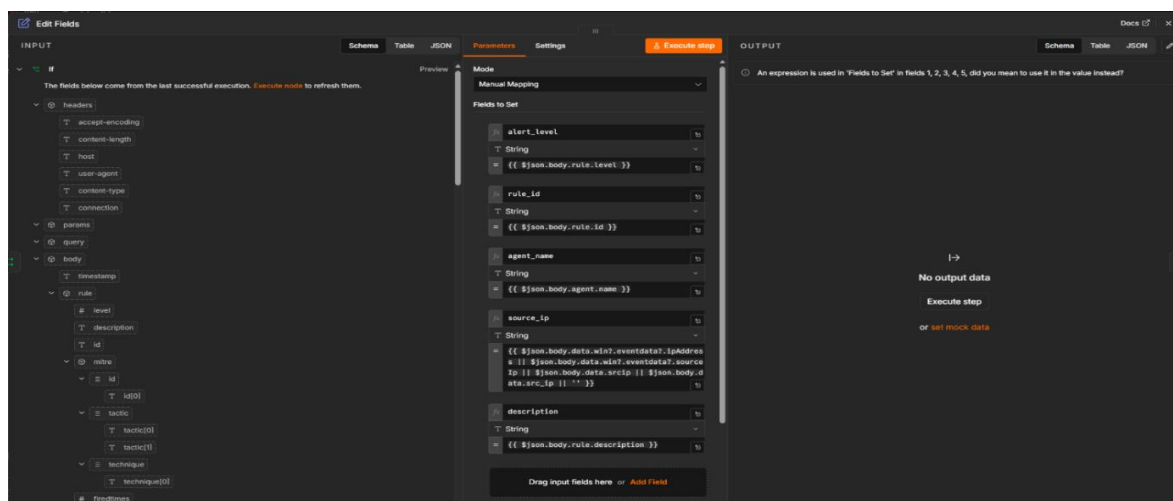


Fig: n8n Edit Fields Configuration

A dedicated Telegram bot create using @BotFather and named MiniCSOC_AlertBot for CSOC security notifications. The bot was connected to the Telegram node in n8n using its bot credential.The bot credential token is intentionally excluded from this project document for security reasons.n8n Telegram credential Base URL: <https://api.telegram.org> From BotFather get Access Token Token format (1234567890:AAxxxxxxxxxxxxxxxxxxxxxxxxxxxxx) The Telegram chat was initialized by sending:

/start

The chat ID was then retrieved using the Telegram Bot API using Ubuntu:

```
curl -s "https://api.telegram.org/bot<NEW_TOKEN>/getUpdates"
```

Response

```
"chat":{"id":1914821583,
```

This Chat ID was used in the n8n Telegram node.

The Telegram node was configured to send a message using the extracted alert fields.

```
Resource      : Message
Operation     : Send Message
Reply Markup  : None
```

The message format used in the final workflow was:

```
🚨 CSOC SECURITY ALERT

Severity: {{$json.alert_level}}
Rule ID: {{$json.rule_id}}
Agent: {{$json.agent_name}}
Source IP: {{$json.source_ip}}

{{$json.description}}
```

This format provides the security analyst with the severity, rule identifier, affected agent, source IP, and event description.

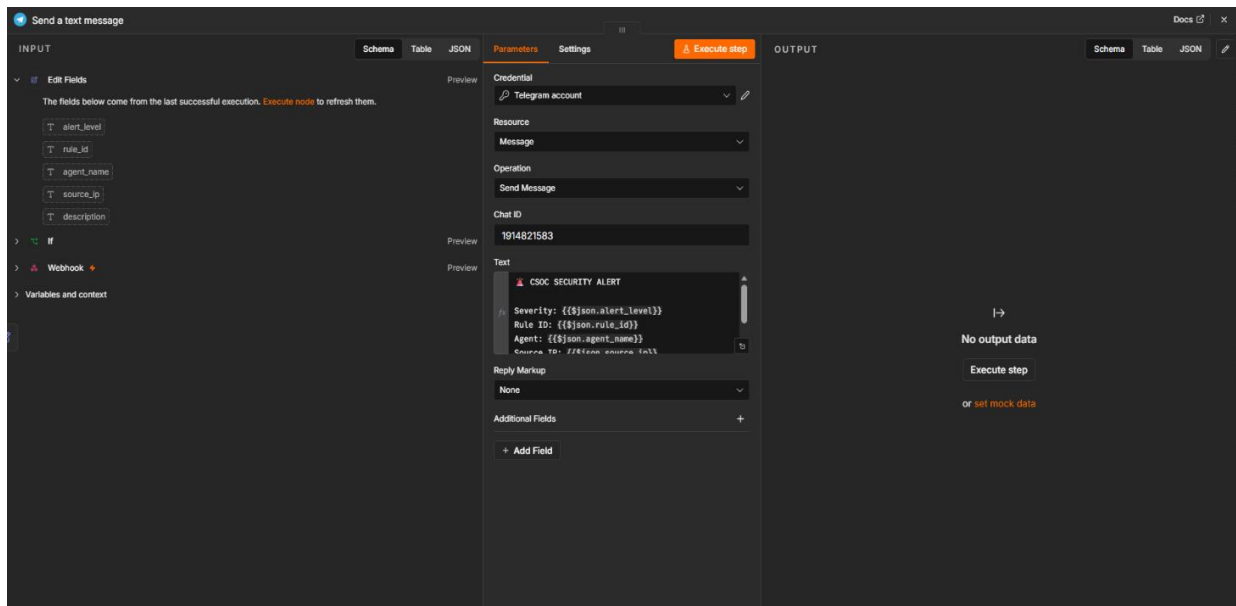


Fig: Telegram Notification Configuration

And telegram account:

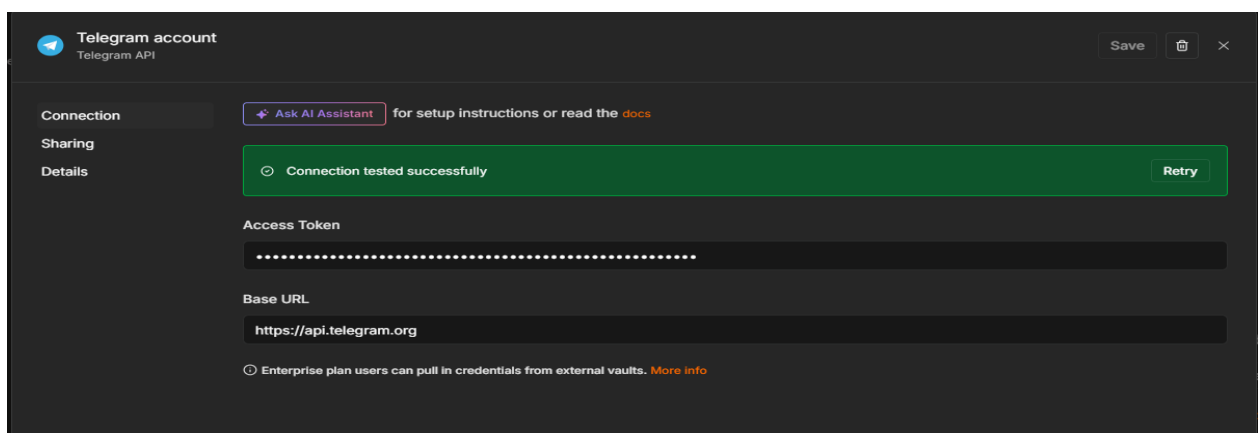


Fig: Telegram Account Configuration

Custom Wazuh-to-n8n Python Integration:

To connect Wazuh directly with the n8n production webhook, a custom Wazuh integration script named custom-n8n was created at:

```
sudo nano /var/ossec/integrations/custom-n8n
```

```
#!/usr/bin/env python3

import sys
import json
import urllib.request

alert_file = sys.argv[1]
hook_url = sys.argv[3]
```

```

with open(alert_file, "r") as f:
    lines = f.readlines()

if not lines:
    sys.exit(0)

alert = json.loads(lines[-1])

data = json.dumps(alert).encode("utf-8")

request = urllib.request.Request(
    hook_url,
    data=data,
    headers={"Content-Type": "application/json"},
    method="POST"
)

try:
    with urllib.request.urlopen(request, timeout=10) as response:
        print("n8n webhook response:", response.status)
except Exception as e:
    print("n8n webhook error:", e)
    sys.exit(1)

```

The final working Python integration script was:

The custom integration script was assigned the required permissions and ownership:

```

sudo chmod 750 /var/ossec/integrations/custom-n8n
sudo chown root:wazuh /var/ossec/integrations/custom-n8n

```

Wazuh Manager Integration Configuration:

Before modifying the Wazuh Manager configuration, a backup of ossec.conf was created:

```

sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.bak-n8n

```

The following integration block was added to the Wazuh Manager configuration file

open

```

sudo nano /var/ossec/etc/ossec.conf

```

added

```

<integration>
  <name>custom-n8n</name>
  <hook_url>http://192.168.0.243:5678/webhook/d988905a-3793-4db8-9d84-
f853ba0a6009</hook_url>
  <level>10</level>
  <alert_format>json</alert_format>
</integration>

```

The level value of 10 ensures that the custom integration processes Wazuh alerts at severity level 10 and above. The Wazuh Manager configuration was validated and the service was restarted:

```
sudo /var/ossec/bin/wazuh-analysisd -t
sudo systemctl restart wazuh-manager
sudo systemctl is-active wazuh-manager
```

The successful service verification output was:

```
active
```

The integration status was also verified in the Wazuh Manager log:

```
sudo grep -i -E "custom-n8n|n8n webhook|integration" /var/ossec/logs/ossec.log | tail
-30
```

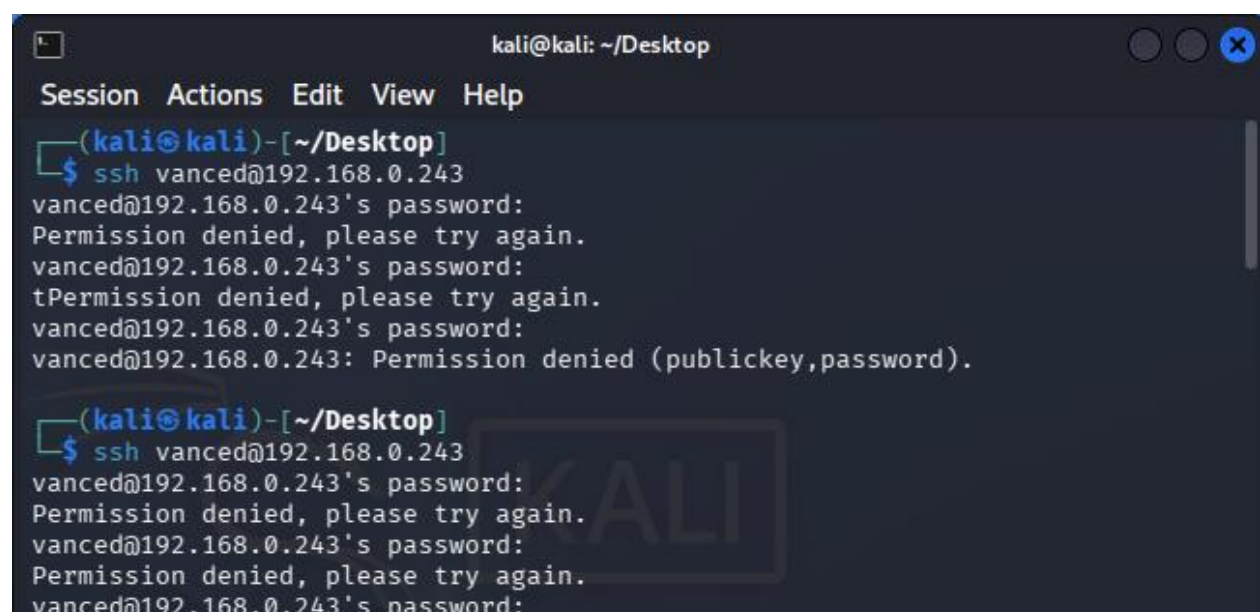
And get

```
wazuh-integratord: INFO: Enabling integration for: 'custom-n8n'.
```

The log confirmed that the custom integration was enabled with the message: "wazuh-integratord: INFO: Enabling integration for: 'custom-n8n'."

Test Execution and Results:

The complete Wazuh → n8n → Telegram automation was tested using the Ubuntu SSH brute-force attack detection scenario. Five failed SSH authentication attempts were generated from Kali against the Ubuntu host:



```
kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ ssh vanced@192.168.0.243
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
tPermission denied, please try again.
vanced@192.168.0.243's password:
vanced@192.168.0.243: Permission denied (publickey,password).
```

```
(kali@kali)-[~/Desktop]
$ ssh vanced@192.168.0.243
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
Permission denied, please try again.
vanced@192.168.0.243's password:
```

Fig: Controlled SSH Brute-Force Simulation from Kali Linux

The test generated failed authentication events and triggered the custom Wazuh SSH brute-force rule 100101 at severity level 12 and Wazuh Active Response Block Event rule 100600 at severity level 10 and also Wazuh Active Response Unblock Event rule 100601 at severity level 10. The final Telegram notification confirmed successful end-to-end delivery:

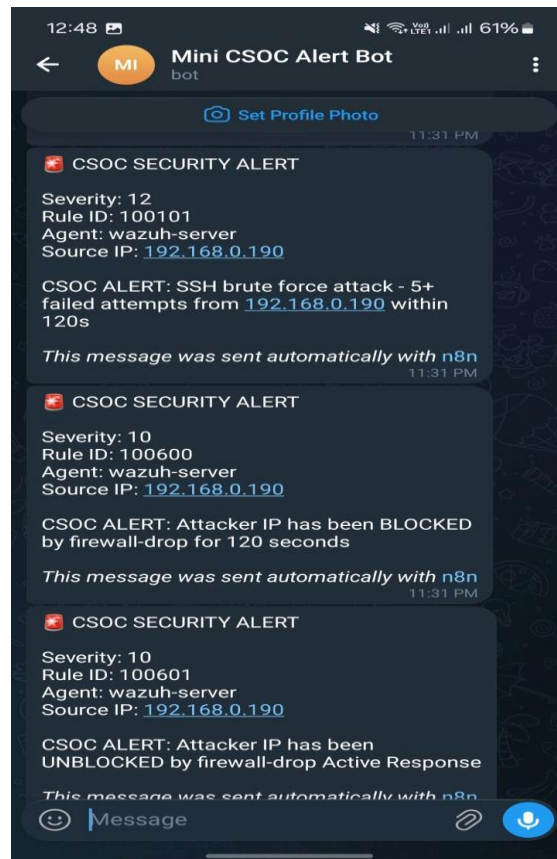


Fig: Telegram Response

Security Dashboards

The Wazuh Dashboard was used as the central visualization and monitoring interface for the Mini Automated CSOC project. The dashboards provide a consolidated view of security alerts, detection events, alert severity, endpoint activity, brute-force attacks, Active Response events, PowerShell activity, and File Integrity Monitoring (FIM) events. Two dedicated dashboards were developed for the project:

- **Mini CSOC Security Overview**
- **CSOC Incident Detection & Response**

These dashboards allow security events generated by the different detection modules to be monitored and analyzed from a centralized interface. The final project documentation confirms that the dashboards were designed to provide both a high-level security overview and an incident-focused investigation view.

Dashboard 1 — Mini CSOC Security Overview:

The Mini CSOC Security Overview dashboard was designed to provide a high-level summary of the security status of the monitored CSOC environment. It aggregates Wazuh alerts and presents the most important security metrics through numerical indicators and visualizations. The dashboard provides visibility into:

1.Total Security Alerts

This panel displays the total number of security alerts generated within the selected dashboard time range.

2. High Severity Alerts

This panel displays the number of alerts generated with higher severity levels. It provides a quick indication of potentially important security activity requiring analyst attention.

3. Alerts by Agent

This visualization shows the distribution of security alerts across the monitored agents.

In this project, the primary monitored systems are:

- wazuh-server
- Windows-Host

4. Top Alert Rules

This panel displays the most frequently triggered Wazuh rules within the selected time range. It helps identify the detection rules generating the highest number of events.

5. Alert Trend Over Time

The alert trend visualization shows how the number of security alerts changes over time. This can help identify periods of increased security activity.

6. Custom CSOC Detection Overview

This panel provides visibility into the project's custom CSOC detection rules, including the major rules developed for authentication, brute-force detection, PowerShell monitoring, and Active Response. The custom detection rules implemented in the project include 100001, 100100, 100101, 100200, 100300, 100600, and 100601. The underlying detection mapping is documented in the consolidated rule configuration.

7. High Severity Alert Levels

This visualization groups alerts according to their Wazuh severity level, allowing the analyst to quickly distinguish lower-severity events from high-severity security alerts.

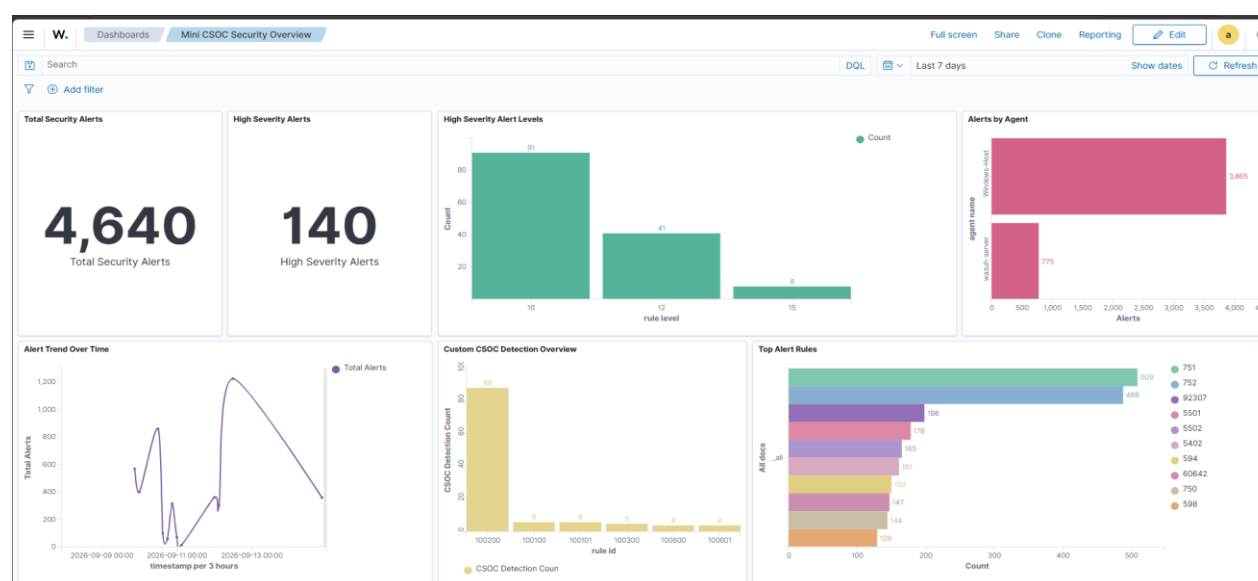


Fig: Mini CSOC Security Overview Dashboard

Dashboard 2 — CSOC Incident Detection & Response:

The CSOC Incident Detection & Response dashboard was developed to provide a more focused view of security incidents and response activities. Unlike the first dashboard, which provides an overall security summary, this dashboard focuses on the project's major detection and response scenarios. The dashboard contains the following monitoring panels:

1. Failed Login Attempts

The Failed Login Attempts panel displays authentication failures detected by Wazuh. For Ubuntu SSH authentication, individual failed authentication events are identified through the Wazuh authentication detection chain and classified by custom Rule 100001 at Level 5. For Windows authentication activity, the built-in Rule 60122 identifies failed Windows logon events, which are subsequently correlated by custom Rule 100100.

2. Brute Force Attacks

The Brute Force Attacks panel displays repeated authentication failures classified as brute-force activity. For Ubuntu SSH, Rule 100101 detects five or more failed authentication attempts from the same source IP within 120 seconds. For Windows, Rule 100100 detects five matching failed logon events associated with the same target username within the configured 120-second window.

3. Active Response

The Active Response panel provides visibility into automated response activity. In the final implementation, Active Response is connected specifically to Ubuntu SSH brute-force Rule 100101. When the rule is triggered, the firewall-drop mechanism blocks the identified source IP for the configured timeout period. The dashboard also displays the corresponding block and automatic-unblock events through custom Rules 100600 and 100601. Rule 100600 reports the IP-blocking event, while Rule 100601 reports the automatic recovery/unblocking event.

4. Suspicious PowerShell Activity

The Suspicious PowerShell Activity panel provides visibility into suspicious PowerShell events detected by the project. Custom Rule 100300 was configured on top of Wazuh Rule 91809 to identify suspicious PowerShell Base64 decoding activity and classify it at Level 12.

5. File Integrity Monitoring

The File Integrity Monitoring panel visualizes file changes detected by Wazuh FIM/Syscheck. The monitored directory was: C:\CSOC-FIM-Test. The project tested three types of file activity: File Creation 554, File Modification 550, File Deletion 553

6. Alert Severity Distribution

The Alert Severity Distribution panel provides a graphical representation of alerts according to their Wazuh severity levels. This visualization allows the analyst to identify the distribution of low-, medium-, and high-severity events within the selected time range.

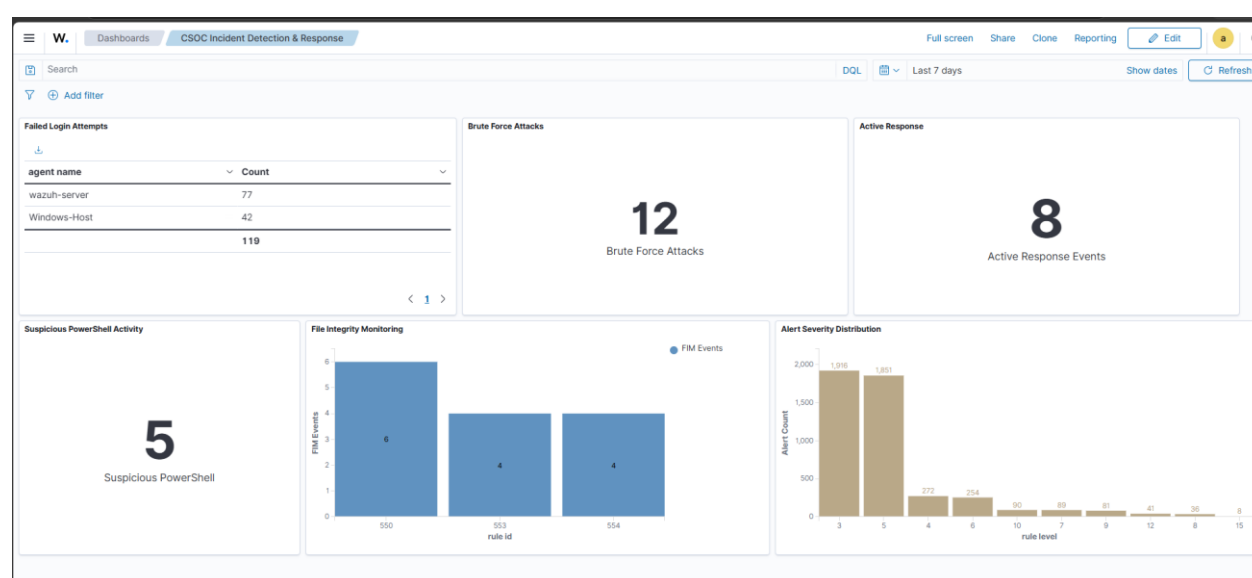


Fig: CSOC Incident Detection & Response Dashboard

Conclusion

The Mini Automated CSOC was successfully implemented and tested in a controlled laboratory environment. Wazuh was used to collect and analyze security events from the monitored systems and to generate alerts for different security activities.

The project successfully detected **SSH authentication failures, SSH brute-force attacks, Windows brute-force activity, Windows Firewall blocked connections, suspicious PowerShell Base64 decoding, and file changes through FIM**. The custom Wazuh rules were tested successfully and produced the expected alerts.

Automatic incident response was also successfully implemented for Ubuntu SSH brute-force attacks. After five failed attempts from the same source IP within 120 seconds, Wazuh automatically blocked the attacker IP for 120 seconds and then automatically unblocked it.

The integration of n8n and Telegram made the alert process more automatic by sending high-severity security alerts directly to Telegram. The Wazuh dashboards also provided a simple and centralized view of the security status and major incidents.

In conclusion, the project successfully demonstrates that a small laboratory-based environment can implement the core functions of a modern CSOC, including **centralized security monitoring, event detection, custom rule-based analysis, automated incident response, alert notification, and security visualization**. The completed system provides a practical foundation for future enhancement with additional detection rules, more monitored endpoints, advanced threat intelligence, broader automated response capabilities, and scalable SOC operations.